

06g — Hashing Algorithms

Key Concepts

- **Hash Function** — one-way function; converts input to fixed-length string, cannot be reversed
- **Salt** — random unique string added to each password before hashing; prevents rainbow table attacks
- **Rainbow Table** — pre-computed hash → password lookup table; defeated by salting
- **Cost Factor** — bcrypt parameter controlling how slow the hash computation is; increase as hardware improves
- **Memory-Hard** — requires large RAM to compute; makes GPU parallelization expensive (scrypt, Argon2)
- **Collision** — two different inputs producing the same hash; breaks MD5 and SHA-1
- **Brute Force** — trying billions of password guesses by computing hashes at speed

Algorithm Comparison

Algorithm	Speed	Salting	Safe for Passwords?	Use For
MD5	~10B/sec on GPU	No	Never	File integrity only
SHA-1	Fast	No	Never	Legacy only (migrate away)
SHA-256	~10B/sec on GPU	No	No	JWT signing, TLS, file integrity
bcrypt	~250/sec on GPU	Built-in	Yes	Passwords (standard choice)
scrypt	Slow + memory-hard	Built-in	Yes	Passwords (GPU-resistant)
Argon2	Configurable + memory-hard	Built-in	Yes	Passwords (current gold standard)

How Hashing Works for Passwords

Login Flow

1. User registers → hash password with bcrypt → store hash in DB
2. User logs in → hash submitted password → compare hashes
3. Original password never stored anywhere

Why Fast Algorithms Fail for Passwords

MD5: GPU computes 10 billion hashes/second
Attacker tries 10B passwords/sec → cracks in seconds

bcrypt (cost 12): GPU computes ~250 hashes/second
Attacker tries 250 passwords/sec → cracks in years

Why Salting Matters

```
Without salt:
  user1: "password123" → abc123hash
  user2: "password123" → abc123hash ← same hash, crack once = crack both

With salt:
  user1: "password123" + "xK92mP" → completely different hash
  user2: "password123" + "q7Lm3R" → completely different hash
  ← same password, different hashes, rainbow tables useless
```

bcrypt, scrypt, Argon2 handle salting automatically.

Use Case Reference

Use Case	Algorithm
Password storage	bcrypt / Argon2
JWT signing	HMAC-SHA256
File integrity check	SHA-256
TLS certificate	SHA-256
Digital signatures	SHA-256 / SHA-3
Old legacy system	SHA-1 (migrate away)
Anything	Never MD5

Industry Examples

- **LinkedIn (2012)** — unsalted SHA1 hashes; 6.5M passwords cracked immediately after breach
 - **Adobe (2013)** — 153M passwords with weak reversible encryption; cracked in days
 - **Dropbox (2016)** — bcrypt used for newer accounts; older accounts with SHA1 were more exposed
-

Interview Questions

Q: Why is MD5 not suitable for passwords? A: MD5 is too fast — GPUs compute 10B hashes/sec, enabling rapid brute force. No built-in salting means identical passwords produce identical hashes, enabling rainbow table attacks.

Q: What is salting and why is it important? A: A random unique value added to each password before hashing. Means identical passwords produce different hashes — defeats rainbow tables and prevents cracking multiple accounts at once.

Q: What is bcrypt's cost factor? A: A configurable parameter that controls how slow the hash is. Higher cost = slower computation. Can be increased as hardware improves to keep password cracking expensive.

Q: What makes Argon2 better than bcrypt? A: Argon2 is memory-hard — requires large RAM amounts, not just CPU cycles. This makes GPU/ASIC parallelization expensive. Won the 2015 Password Hashing Competition.

Q: When would you use SHA-256 vs bcrypt? A: SHA-256 for data integrity, JWT signing, certificates — cases needing fast verification. bcrypt for passwords — where speed is the enemy and slowness is the security feature.